COMPETITION MASTER 01 — المزرعة الذكية الكاملة على Raspberry Pi 5

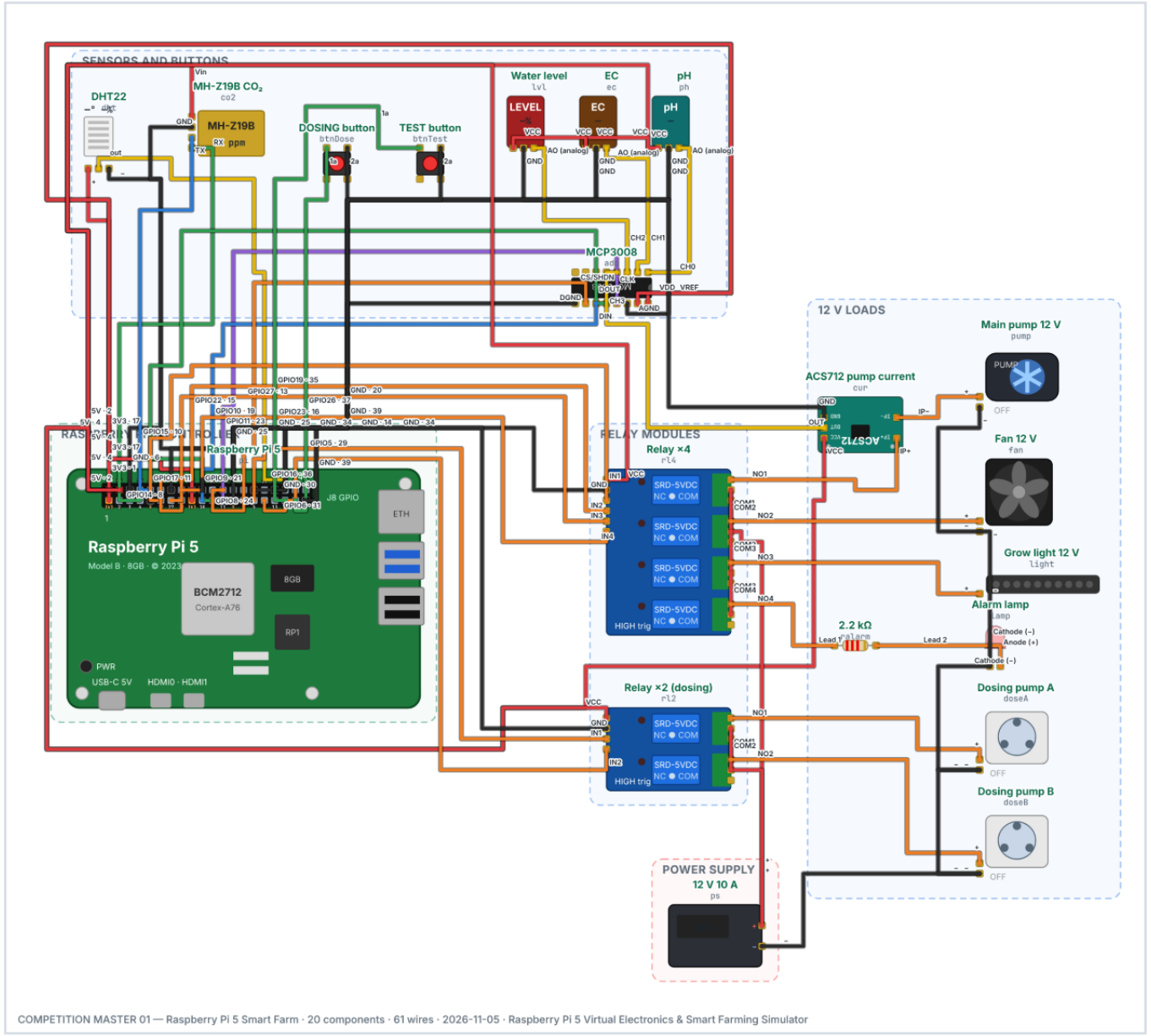
ملف الدراسة الكامل

قراءة ستة حساسات، وتشغيل أوضاع AUTO وDOSING وTEST مع الحماية والإنذارات، وعرض كل شيء في الطرفية وفي صفحة ويب Flask بسيطة.

المتحكم	Raspberry Pi 5 Model B
اللغة	Python 3 (gpiozero, pyserial, adafruit_dht, Flask)
ملف البرنامج	main.py
عرض الحالة	الطرفية (أسطر STATUS) + تبويب Web (صفحة Flask على المنفذ 5000).
القطع / الأسلاك	20 / 61
تاريخ التصدير	2026-11-05

DHT22 (GPIO16)، وMH-Z19B (UART0)، وMCP3008 (SPI0: pH وEC والمستوى وACS712)، ومرحّل 4 قنوات + مرحّل قناتين، وأحمال 12 فولت، وزرّان.

1. مخطط الدائرة



20 قطعة · 61 سلك

2. جدول التوصيلات — شرح كل سلك

#	من	إلى	النوع	لماذا	لو انفصل
1	Raspberry Pi 5 · 3V3	DHT22 · +	طاقة	تغذية DHT22	يفقد DHT22 التغذية — لا رد من DHT22: الحرارة None، تضيع القراءة: إنذار SENSOR ERROR ويضيء مصباح الإنذار، والمروحة تعمل للأمان
2	Raspberry Pi 5 · GPIO16	DHT22 · out	بيانات رقمية	DHT22: خط بيانات رقمي واحد إلى Pi	لا رد من DHT22: الحرارة None، تضيع القراءة: إنذار SENSOR ERROR ويضيء مصباح الإنذار، والمروحة تعمل للأمان
3	DHT22 · –	Raspberry Pi 5 · GND	أرضي	الأرضي (مسار العودة) لـDHT22	يفقد DHT22 التغذية — لا رد من DHT22: الحرارة None، تضيع القراءة: إنذار SENSOR ERROR ويضيء مصباح الإنذار، والمروحة تعمل للأمان
4	Raspberry Pi 5 · 5V	MH-Z19B CO_2 · Vin	طاقة	تغذية حساس MH-Z19B CO_2	يفقد حساس MH-Z19B CO_2 التغذية — لا رد على الأمر 0x86: co2 = None، تضيع القراءة: إنذار SENSOR ERROR ويضيء مصباح الإنذار
5	MH-Z19B CO_2 · GND	Raspberry Pi 5 · GND	أرضي	الأرضي (مسار العودة) لـحساس MH-Z19B CO_2	يفقد حساس MH-Z19B CO_2 التغذية — لا رد على الأمر 0x86: co2 = None، تضيع القراءة: إنذار SENSOR ERROR ويضيء مصباح الإنذار
6	Raspberry Pi 5 · GPIO14	MH-Z19B CO_2 · RX	بيانات رقمية	حساس CO_2 MH-Z19B: UART TX: الحساس ← RX المتحكم، وRX الحساس ← TX المتحكم (متقاطعان)	لا رد على الأمر 0x86: co2 = None، تضيع القراءة: إنذار SENSOR ERROR ويضيء مصباح الإنذار
7	MH-Z19B CO_2 · TX	Raspberry Pi 5 · GPIO15	بيانات رقمية	حساس CO_2 MH-Z19B: UART TX: الحساس ← RX المتحكم، وRX الحساس ← TX المتحكم (متقاطعان)	لا رد على الأمر 0x86: co2 = None، تضيع القراءة: إنذار SENSOR ERROR ويضيء مصباح الإنذار
8	Raspberry Pi 5 · 3V3	MCP3008 · VDD	طاقة	تغذية محوّل MCP3008	يفقد محوّل MCP3008 التغذية — تصبح القراءات التناظرية الأربع خاطئة (pH وEC والمستوى والتيار): LOW WATER أو SENSOR ERROR، ويُمنع الري
9	Raspberry Pi 5 · 3V3	MCP3008 · VREF	طاقة	تغذية محوّل MCP3008	يفقد محوّل MCP3008 التغذية — تصبح القراءات التناظرية الأربع خاطئة (pH وEC والمستوى والتيار): LOW WATER أو SENSOR ERROR، ويُمنع الري
10	MCP3008 · AGND	Raspberry Pi 5 · GND	أرضي	الأرضي (مسار العودة) لـمحوّل MCP3008	يفقد محوّل MCP3008 التغذية — تصبح القراءات التناظرية الأربع خاطئة (pH وEC والمستوى والتيار): LOW WATER أو SENSOR ERROR، ويُمنع الري
11	MCP3008 · DGND	Raspberry Pi 5 · GND	أرضي	الأرضي (مسار العودة) لـمحوّل MCP3008	يفقد محوّل MCP3008 التغذية — تصبح القراءات التناظرية الأربع خاطئة (pH وEC والمستوى والتيار): LOW WATER أو SENSOR ERROR، ويُمنع الري
12	Raspberry Pi 5 · GPIO11	MCP3008 · CLK	بيانات رقمية	محوّل MCP3008: ناقل SPI: لا توجد مداخل تناظرية في Pi، فيقيس MCP3008 الحساسات التناظرية الأربعة	تصبح القراءات التناظرية الأربع خاطئة (pH وEC والمستوى والتيار): LOW WATER أو SENSOR ERROR، ويُمنع الري
13	MCP3008 · DOUT	Raspberry Pi 5 · GPIO9	بيانات رقمية	محوّل MCP3008: ناقل SPI: لا توجد مداخل تناظرية في Pi، فيقيس MCP3008 الحساسات التناظرية الأربعة	تصبح القراءات التناظرية الأربع خاطئة (pH وEC والمستوى والتيار): LOW WATER أو SENSOR ERROR، ويُمنع الري
14	MCP3008 · DIN	Raspberry Pi 5 · GPIO10	بيانات رقمية	محوّل MCP3008: ناقل SPI: لا توجد مداخل تناظرية في Pi، فيقيس MCP3008 الحساسات التناظرية الأربعة	تصبح القراءات التناظرية الأربع خاطئة (pH وEC والمستوى والتيار): LOW WATER أو SENSOR ERROR، ويُمنع الري
15	Raspberry Pi 5 · GPIO8	MCP3008 · CS/SHDN	بيانات رقمية	محوّل MCP3008: ناقل SPI: لا توجد مداخل تناظرية في Pi، فيقيس MCP3008 الحساسات التناظرية الأربعة	تصبح القراءات التناظرية الأربع خاطئة (pH وEC والمستوى والتيار): LOW WATER أو SENSOR ERROR، ويُمنع الري
16	Raspberry Pi 5 · 5V	pH · VCC	طاقة	تغذية وحدة حساس pH	يفقد وحدة حساس pH التغذية — تُقرأ pH خطأ (قريبة من 0 ← أقل من 1): SENSOR ERROR
17	pH · GND	Raspberry Pi 5 · GND	أرضي	الأرضي (مسار العودة) لـوحدة حساس pH	يفقد وحدة حساس pH التغذية — تُقرأ pH خطأ (قريبة من 0 ← أقل من 1): SENSOR ERROR
18	pH · VCC	EC · VCC	طاقة	تغذية وحدة حساس pH	يفقد وحدة حساس pH التغذية — تُقرأ pH خطأ (قريبة من 0 ← أقل من 1): SENSOR ERROR
19	EC · GND	pH · GND	أرضي	الأرضي (مسار العودة) لـوحدة حساس pH	يفقد وحدة حساس pH التغذية — تُقرأ pH خطأ (قريبة من 0 ← أقل من 1): SENSOR ERROR
20	EC · VCC	Water level · VCC	طاقة	تغذية وحدة حساس EC	يفقد وحدة حساس EC التغذية — تُقرأ EC خطأ (قريبة من 0): OUT OF RANGE

#	من	إلى	النوع	لماذا	لو انفصل
21	Water level · GND	EC · GND	أرضي	الأرضي (مسار العودة) لـوحدة حساس EC	يفقد وحدة حساس EC التغذية — تُقرأ EC خطأً (قريبة من 0): OUT OF RANGE
22	pH · AO (analog)	MCP3008 · CH0	إشارة تناظرية	وحدة حساس pH: جهد تناظري: pH = 3.5 × V	تُقرأ pH خطأً (قريبة من 0 ← أقل من 1): SENSOR ERROR
23	EC · AO (analog)	MCP3008 · CH1	إشارة تناظرية	وحدة حساس EC: جهد تناظري: EC = V ÷ 0.5	تُقرأ EC خطأً (قريبة من 0): OUT OF RANGE
24	Water level · AO (analog)	MCP3008 · CH2	إشارة تناظرية	حساس مستوى الماء: جهد تناظري: 3 فولت = 100٪	يُقرأ المستوى خطأً (قريباً من 0٪): LOW WATER ويُرفض الري والجرعات
25	Raspberry Pi 5 · 5V	ACS712 pump current · VCC	طاقة	تغذية حساس التيار ACS712	يفقد حساس التيار ACS712 التغذية — يُقرأ تيار المضخة خطأً: الري التالي يبلّغ عن PUMP FAILURE
26	ACS712 pump current · GND	Raspberry Pi 5 · GND	أرضي	الأرضي (مسار العودة) لـحساس التيار ACS712	يفقد حساس التيار ACS712 التغذية — يُقرأ تيار المضخة خطأً: الري التالي يبلّغ عن PUMP FAILURE
27	ACS712 pump current · OUT	MCP3008 · CH3	إشارة تناظرية	حساس التيار ACS712: جهد تناظري: 2.5 فولت + 0.185 فولت لكل أمبير من تيار المضخة	يُقرأ تيار المضخة خطأً: الري التالي يبلّغ عن PUMP FAILURE
28	Raspberry Pi 5 · 5V	Relay ×4 · VCC	طاقة	تغذية وحدة مرحّل 4 قنوات	تفقد ملفات المرحّل التغذية: لا تعمل المضخة ولا المروحة ولا الإضاءة ولا مصباح الإنذار
29	Relay ×4 · GND	Raspberry Pi 5 · GND	أرضي	الأرضي (مسار العودة) لـوحدة مرحّل 4 قنوات	تفقد ملفات المرحّل التغذية: لا تعمل المضخة ولا المروحة ولا الإضاءة ولا مصباح الإنذار
30	Raspberry Pi 5 · GPIO17	Relay ×4 · IN1	بيانات رقمية	وحدة مرحّل 4 قنوات: مدخل المرحّل: المستوى HIGH يغلق القناة	لا يتبع المرحّل البرنامج: يطبع البرنامج pump=ON بينما يبقى الحمل متوقفاً
31	Raspberry Pi 5 · GPIO22	Relay ×4 · IN2	بيانات رقمية	وحدة مرحّل 4 قنوات: مدخل المرحّل: المستوى HIGH يغلق القناة	لا يتبع المرحّل البرنامج: يطبع البرنامج pump=ON بينما يبقى الحمل متوقفاً
32	Raspberry Pi 5 · GPIO27	Relay ×4 · IN3	بيانات رقمية	وحدة مرحّل 4 قنوات: مدخل المرحّل: المستوى HIGH يغلق القناة	لا يتبع المرحّل البرنامج: يطبع البرنامج pump=ON بينما يبقى الحمل متوقفاً
33	Raspberry Pi 5 · GPIO23	Relay ×4 · IN4	بيانات رقمية	وحدة مرحّل 4 قنوات: مدخل المرحّل: المستوى HIGH يغلق القناة	لا يتبع المرحّل البرنامج: يطبع البرنامج pump=ON بينما يبقى الحمل متوقفاً
34	Raspberry Pi 5 · 5V	Relay ×2 (dosing) · VCC	طاقة	تغذية وحدة مرحّل قناتين (الجرعات)	يفقد مرحّلا الجرعات التغذية: لا جرعات أبداً
35	Relay ×2 (dosing) · GND	Raspberry Pi 5 · GND	أرضي	الأرضي (مسار العودة) لـوحدة مرحّل قناتين (الجرعات)	يفقد مرحّلا الجرعات التغذية: لا جرعات أبداً
36	Raspberry Pi 5 · GPIO5	Relay ×2 (dosing) · IN1	بيانات رقمية	وحدة مرحّل قناتين (الجرعات): مدخل المرحّل: المستوى HIGH يغلق القناة	لا تعمل مضخة الجرعات رغم أن البرنامج يطبع DOSING ...
37	Raspberry Pi 5 · GPIO6	Relay ×2 (dosing) · IN2	بيانات رقمية	وحدة مرحّل قناتين (الجرعات): مدخل المرحّل: المستوى HIGH يغلق القناة	لا تعمل مضخة الجرعات رغم أن البرنامج يطبع DOSING ...
38	ACS712 pump current · IP−	Main pump 12 V · +	دائرة الحمل (12 فولت)	دائرة 12 فولت لـالمضخة الرئيسية: موجب المصدر ← تلامس المرحّل ← الحمل ← سالب المصدر	لا تعمل المضخة: لا تيار في ACS712 ← PUMP FAILURE: no current
39	2.2 kΩ · Lead 2	Alarm lamp · Anode (+)	بيانات رقمية	مصباح الإنذار: مصباح 12 فولت (LED + 2.2 كيلو أوم)	تُطبع الإنذارات لكن المصباح لا يضيء أبداً
40	12 V 10 A · +	Relay ×4 · COM1	دائرة الحمل (12 فولت)	دائرة 12 فولت لـالمضخة الرئيسية: موجب المصدر ← تلامس المرحّل ← الحمل ← سالب المصدر	لا تعمل المضخة: لا تيار في ACS712 ← PUMP FAILURE: no current
41	Relay ×4 · NO1	ACS712 pump current · IP+	دائرة الحمل (12 فولت)	دائرة 12 فولت لـالمضخة الرئيسية: موجب المصدر ← تلامس المرحّل ← الحمل ← سالب المصدر	لا تعمل المضخة: لا تيار في ACS712 ← PUMP FAILURE: no current
42	Relay ×4 · COM1	Relay ×4 · COM2	دائرة الحمل (12 فولت)	دائرة 12 فولت لـالمروحة: موجب المصدر ← تلامس المرحّل ← الحمل ← سالب المصدر	لا تعمل المروحة: يستمر هواء المزرعة بالسخونة، والبرنامج لا يعرف ذلك (لا توجد تغذية راجعة للمروحة)
43	Relay ×4 · NO2	Fan 12 V · +	دائرة الحمل (12 فولت)	دائرة 12 فولت لـالمروحة: موجب المصدر ← تلامس المرحّل ← الحمل ← سالب المصدر	لا تعمل المروحة: يستمر هواء المزرعة بالسخونة، والبرنامج لا يعرف ذلك (لا توجد تغذية راجعة للمروحة)
44	Relay ×4 · COM2	Relay ×4 · COM3	دائرة الحمل (12 فولت)	دائرة 12 فولت لـإضاءة النمو: موجب المصدر ← تلامس المرحّل ← الحمل ← سالب المصدر	تبقى الإضاءة مطفأة رغم أن البرنامج يطبع light=ON
45	Relay ×4 · NO3	Grow light 12 V · +	دائرة الحمل (12 فولت)	دائرة 12 فولت لـإضاءة النمو: موجب المصدر ← تلامس المرحّل ← الحمل ← سالب المصدر	تبقى الإضاءة مطفأة رغم أن البرنامج يطبع light=ON

#	من	إلى	النوع	لماذا	لو انفصل
46	Relay ×4 · COM3	Relay ×4 · COM4	دائرة الحمل (12 فولت)	دائرة 12 فولت لـمصباح الإنذار: موجب المصدر ← تلامس المرحّل ← الحمل ← سالب المصدر	تُطبع الإنذارات لكن المصباح لا يضيء أبداً
47	Relay ×4 · NO4	2.2 kΩ · Lead 1	دائرة الحمل (12 فولت)	دائرة 12 فولت لـمصباح الإنذار: موجب المصدر ← تلامس المرحّل ← الحمل ← سالب المصدر	تُطبع الإنذارات لكن المصباح لا يضيء أبداً
48	12 V 10 A · +	Relay ×2 (dosing) · COM1	دائرة الحمل (12 فولت)	دائرة 12 فولت لـمضخة الجرعات A: موجب المصدر ← تلامس المرحّل ← الحمل ← سالب المصدر	لا تُضخ الجرعة A رغم طباعة DOSING A (تبقى مجاميع المزرعة 0 مل)
49	Relay ×2 (dosing) · NO1	Dosing pump A · +	دائرة الحمل (12 فولت)	دائرة 12 فولت لـمضخة الجرعات A: موجب المصدر ← تلامس المرحّل ← الحمل ← سالب المصدر	لا تُضخ الجرعة A رغم طباعة DOSING A (تبقى مجاميع المزرعة 0 مل)
50	Relay ×2 (dosing) · COM1	Relay ×2 (dosing) · COM2	دائرة الحمل (12 فولت)	دائرة 12 فولت لـمضخة الجرعات B: موجب المصدر ← تلامس المرحّل ← الحمل ← سالب المصدر	لا تُضخ الجرعة B رغم طباعة DOSING B
51	Relay ×2 (dosing) · NO2	Dosing pump B · +	دائرة الحمل (12 فولت)	دائرة 12 فولت لـمضخة الجرعات B: موجب المصدر ← تلامس المرحّل ← الحمل ← سالب المصدر	لا تُضخ الجرعة B رغم طباعة DOSING B
52	Main pump 12 V · –	Fan 12 V · –	دائرة الحمل (12 فولت)	دائرة 12 فولت لـالمضخة الرئيسية: موجب المصدر ← تلامس المرحّل ← الحمل ← سالب المصدر	لا تعمل المضخة: لا تيار في ACS712 ← PUMP FAILURE: no current
53	Fan 12 V · –	Grow light 12 V · –	دائرة الحمل (12 فولت)	دائرة 12 فولت لـالمروحة: موجب المصدر ← تلامس المرحّل ← الحمل ← سالب المصدر	لا تعمل المروحة: يستمر هواء المزرعة بالسخونة، والبرنامج لا يعرف ذلك (لا توجد تغذية راجعة للمروحة)
54	Grow light 12 V · –	Alarm lamp · Cathode (–)	دائرة الحمل (12 فولت)	دائرة 12 فولت لـإضاءة النمو: موجب المصدر ← تلامس المرحّل ← الحمل ← سالب المصدر	تبقى الإضاءة مطفأة رغم أن البرنامج يطبع light=ON
55	Alarm lamp · Cathode (–)	Dosing pump A · –	دائرة الحمل (12 فولت)	دائرة 12 فولت لـمضخة الجرعات A: موجب المصدر ← تلامس المرحّل ← الحمل ← سالب المصدر	لا تُضخ الجرعة A رغم طباعة DOSING A (تبقى مجاميع المزرعة 0 مل)
56	Dosing pump A · –	Dosing pump B · –	دائرة الحمل (12 فولت)	دائرة 12 فولت لـمضخة الجرعات A: موجب المصدر ← تلامس المرحّل ← الحمل ← سالب المصدر	لا تُضخ الجرعة A رغم طباعة DOSING A (تبقى مجاميع المزرعة 0 مل)
57	Dosing pump B · –	12 V 10 A · –	أرضي	الأرضي (مسار العودة) لـمضخة الجرعات B	يفقد مضخة الجرعات B التغذية — لا تُضخ الجرعة B رغم طباعة DOSING B
58	Raspberry Pi 5 · GPIO26	DOSING button · 1a	بيانات رقمية	زر DOSING: مدخل بمقاومة سحب داخلية: مضغوط = موصول بالأرضي = 0	لا يعمل الزر (مقاومة السحب تبقي المدخل "غير مضغوط")
59	DOSING button · 2a	Raspberry Pi 5 · GND	أرضي	الأرضي (مسار العودة) لـزر DOSING	يفقد زر DOSING التغذية — لا يعمل الزر (مقاومة السحب تبقي المدخل "غير مضغوط")
60	Raspberry Pi 5 · GPIO19	TEST button · 1a	بيانات رقمية	زر TEST: مدخل بمقاومة سحب داخلية: مضغوط = موصول بالأرضي = 0	لا يعمل الزر (مقاومة السحب تبقي المدخل "غير مضغوط")
61	TEST button · 2a	Raspberry Pi 5 · GND	أرضي	الأرضي (مسار العودة) لـزر TEST	يفقد زر TEST التغذية — لا يعمل الزر (مقاومة السحب تبقي المدخل "غير مضغوط")

3. قائمة القطع

#	المكون	الموديل / المرجع	الكمية	الخصائص المهمة	الجهد	مستوى المحاكاة	المعرّفات	ملاحظات التركيب
1	Raspberry Pi 5 Model B (8GB)	Raspberry Pi 5 Model B 8GB · Raspberry Pi Ltd	1	1-Wire (dtoverlay=w1-gpio): no; UART0 enabled (dtparam=uart0=on): yes	5 V USB-C in, 3.3 V GPIO logic	محاكاة كاملة	pi	
2	DHT22 temperature/humidity	DHT22 / AM2302 module · Aosong	1	Module with pull-up resistor: yes	3.3–6 V	محاكاة كاملة	dht	
3	MH-Z19B CO_2 sensor	MH-Z19B · Winsen	1	—	4.5–5.5 V supply, 3.3 V UART	محاكاة كاملة	co2	
4	MCP3008 8-channel 10-bit ADC	MCP3008-I/P · Microchip	1	—	2.7–5.5 V	محاكاة كاملة	adc	
5	pH sensor (analog)	Analog pH probe + board (SEN0161-style)	1	—	3.3–5 V	نموذج وظيفي	ph	
6	EC conductivity sensor (analog)	Analog EC probe + board	1	—	3.3–5 V	نموذج وظيفي	ec	
7	Water level sensor (analog)	Analog water-level sensor	1	—	3.3–5 V	نموذج وظيفي	lvl	
8	ACS712 current sensor (5 A)	ACS712ELCTR-05B · Allegro	1	Version: 5 A (185 mV/A)	5 V supply	محاكاة كاملة	cur	
9	Relay module	SRD-05VDC-SL-C opto relay module	1	Channels: 4; Trigger: Active-HIGH (IN=1 → ON); Coil voltage: 5 V	5 V or 12 V coil, 3.3/5 V logic	محاكاة كاملة	rl4	
10	Relay module	SRD-05VDC-SL-C opto relay module	1	Channels: 2; Trigger: Active-HIGH (IN=1 → ON); Coil voltage: 5 V	5 V or 12 V coil, 3.3/5 V logic	محاكاة كاملة	rl2	
11	DC power supply	Bench / adapter DC supply	1	Voltage: 12 V; Current limit: 10 A	—	محاكاة كاملة	ps	
12	Water pump (12 V DC)	12 V DC pump	1	Rated power: 6 W	12 V	نموذج وظيفي	pump	
13	Cooling fan (DC)	12 V DC fan	1	Rated power: 2.4 W	12 V	نموذج وظيفي	fan	
14	LED grow light	12 V LED grow bar	1	Rated power: 36 W	12 V	نموذج وظيفي	light	
15	Resistor	¼ W carbon-film resistor	1	Resistance: 2.2 kΩ	—	محاكاة كاملة	ralarm	
16	LED	5 mm LED	1	Colour: red	Vf ≈ 1.9–2.9 V, 20 mA max	محاكاة كاملة	lamp	
17	Peristaltic dosing pump	12 V peristaltic pump	2	Rated power: 3.6 W	12 V	نموذج وظيفي	doseA, doseB	
18	Push button	6×6 mm tactile switch	2	—	—	محاكاة كاملة	btnDose, btnTest	
19	أسلاك توصيل	—	61	16× أحمر, 5× أصفر, 18× أسود, 4× أخضر, 2× أزرق, 1× بنفسجي, 15× برتقالي	—	—		استخدم أسلاك ذكر–أنثى لموصل Pi، واجعل أسلاك التغذية قصيرة.

4. الإعدادات (الثوابت في أعلى البرنامج)

الاسم	القيمة	المعنى
MAX_TEMP	30	فوق 30 درجة: تشغيل المروحة
FAN_OFF_TEMP	27	تحت 27 درجة: إيقاف المروحة
MIN_WATER	20	أقل مستوى ماء مسموح (%) لتشغيل المضخة
PUMP_SECONDS	5	مدة الري بالثواني
IRRIGATION_HOURS	[7, 12, 17]	ساعات الري في اليوم
LIGHT_ON_HOUR	6	ساعة تشغيل الإضاءة
LIGHT_OFF_HOUR	20	ساعة إطفاء الإضاءة
DOSE_A_ML	10	جرعة السماد A بالمليلتر
DOSE_B_ML	5	جرعة السماد B بالمليلتر
FLOW_ML_PER_SEC	2	تدفق مضخة الجرعات (يحتاج معايرة على الجهاز الحقيقي)
PH_MIN	5.5	أقل pH مقبول
PH_MAX	6.5	أعلى pH مقبول
EC_MIN	1.0	أقل EC مقبول (mS/cm)
EC_MAX	2.5	أعلى EC مقبول
MIN_PUMP_AMPS	0.2	أقل تيار يثبت أن المضخة تعمل فعلاً (أمبير)
TEST_SECONDS	2	مدة تشغيل كل جهاز في وضع الاختبار

قيم تدريبية يمكنك تغييرها، وليست إعدادات معتمدة لكل المحاصيل. دقة الجرعات تعتمد على معايرة المضخات الحقيقية.

5. البرنامج في 9 أقسام

1 · التهيئة والتوصيلات

الإعدادات أولاً، ثم سطر لكل جهاز: لكل حساس ومرحّل اسم ورقم GPIO أو قناة ADC. بقية البرنامج تستخدم هذه الأسماء فقط.

- غيّر الحد هنا وسيتبعه البرنامج كله.
- أرقام GPIO هي أرقام BCM (GPIO17 = الرجل الفعلية 11).

main.py

```
---------- الإعدادات: قيم تدريبية يمكنك تغييرها (1 ---------- #  11
فوق 30 درجة: تشغيل المروحة #                 MAX_TEMP = 30  12
تحت 27 درجة: إيقاف المروحة #              FAN_OFF_TEMP = 27  13
أقل مستوى ماء مسموح (%) لتشغيل المضخة #                 MIN_WATER = 20  14
مدة الري بالثواني #               PUMP_SECONDS = 5  15
ساعات الري في اليوم #  IRRIGATION_HOURS = [7, 12, 17]  16
ساعة تشغيل الإضاءة #              LIGHT_ON_HOUR = 6  17
ساعة إطفاء الإضاءة #            LIGHT_OFF_HOUR = 20  18
جرعة السماد A بالمليلتر #                 DOSE_A_ML = 10  19
جرعة السماد B بالمليلتر #                  DOSE_B_ML = 5  20
تدفق مضخة الجرعات (يحتاج معايرة على الجهاز الحقيقي) #           FLOW_ML_PER_SEC = 2  21
أقل pH مقبول #                  PH_MIN = 5.5  22
أعلى pH مقبول #                   PH_MAX = 6.5  23
أقل EC مقبول (mS/cm) #                  EC_MIN = 1.0  24
أعلى EC مقبول #                  EC_MAX = 2.5  25
أعلى CO2 مقبول (ppm) #                CO2_MAX = 1500  26
أقل تيار يثبت أن المضخة تعمل فعلاً (أمبير) #           MIN_PUMP_AMPS = 0.2  27
مدة تشغيل كل جهاز في وضع الاختبار #              TEST_SECONDS = 2  28
 29
 30  # >>> PINS
---------- التوصيلات: كل جهاز على رجله (2 ---------- #  31
البيانات على GPIO16 :dht = adafruit_dht.DHT22(board.D16)                # DHT22  32
co2_port = serial.Serial("/dev/serial0", 9600, timeout=1)  # MH-Z19B  33
حساس pH على قناة ADC رقم 0 #                         ph_adc = MCP3008(channel=0)  34
حساس EC على القناة 1 #                      ec_adc = MCP3008(channel=1)  35
حساس مستوى الماء على القناة 2 #                      level_adc = MCP3008(channel=2)  36
حساس تيار المضخة على القناة 3 #                           current_adc = MCP3008(channel=3)  37
مرحّل مضخة الري على GPIO17 #                        pump = OutputDevice(17)  38
```

```
GPIO22 مرحّل المروحة على #                                  fan = OutputDevice(22)  39
GPIO27 مرحّل الإضاءة على #                                light = OutputDevice(27)  40
GPIO23 مرحّل مصباح الإنذار على #                          alarm_lamp = OutputDevice(23)  41
GPIO5 مرحّل مضخة الجرعات A على #                               dose_a = OutputDevice(5)  42
GPIO6 مرحّل مضخة الجرعات B على #                               dose_b = OutputDevice(6)  43
(GND إلى) GPIO26 زر وضع الجرعات على #                      dosing_button = Button(26)  44
(GND إلى) GPIO19 زر وضع الاختبار على #                       test_button = Button(19)  45
 46  # <<< PINS
```

2 · قراءة الحساسات

دالة واحدة تقرأ كل شيء وتحفظه في متغيرات. القراءة المفقودة تصبح None/null حتى تراها الإنذارات. الحساسات التناظرية تعطي جهداً؛ ومعادلة لكل حساس تحوّل الجهد إلى pH وEC ونسبة مئوية وأمبير.

- pH = 3.5 + V × 0.5 · EC = V ÷ 1.5 · المستوى = 100 × 3 ÷ V · التيار = (V − 2.5) ÷ 0.185 (وحدات المحاكي؛ عايِر الحساسات الحقيقية).
- يحتاج DHT22 ثانيتين بين القراءات: البرنامج يقرؤه مرة كل ثانيتين على الأكثر.

main.py

```
UART عبر MH-Z19B من CO2 قراءة #                                    :()def read_co2  64
(9 بايت) أمر طلب القراءة #  co2_port.write(bytes([0xFF, 0x01, 0x86, 0, 0, 0, 0, 0, 0x79]))      65
9 بايت من رداً ننتظر #                          reply = co2_port.read(9)      66
الحساس مفصول = لا يوجد رد #                                 :if len(reply) != 9      67
return None تعني: لا توجد قراءة                                      # None          68
ppm التركيز = 3 و 2 البايت #                  return reply[2] * 256 + reply[3]      69
 72  # >>> SENSORS
قراءة كل الحساسات #                                      :()def read_sensors  73
سنغيّر هذه #  global temperature, humidity, co2, ph, ec, water_level, pump_amps, last_dht_time      74
↪ المتغيرات العامة
قراءة جديدة كل ثانيتين فقط :if time.time() - last_dht_time >= 2:            # DHT22      75
نحفظ وقت هذه القراءة #                  ()last_dht_time = time.time          76
نحاول القراءة #                                        :try          77
C° الحرارة #           temperature = dht.temperature              78
% الرطوبة #                humidity = dht.humidity              79
(الحساس مفصول) فشل القراءة #                          :except RuntimeError          80
لا توجد قراءة حرارة #                      temperature = None              81
ولا رطوبة #                          humidity = None              82
(None أو) co2 = read_co2()                                  # CO2 ppm      83
الجهد × ph = round(ph_adc.voltage * 3.5, 2)                # pH = 3.5      84
0.5 ÷ الجهد = ec = round(ec_adc.voltage / 0.5, 2)                 # EC      85
%100 = فولت water_level = round(level_adc.voltage / 3.0 * 100)  # 3      86
صفر أمبير = فولت pump_amps = round((current_adc.voltage - 2.5) / 0.185, 2)  # ACS712: 2.5      87
 88  # <<< SENSORS
```

3 · تشغيل المخرجات

لا يستطيع GPIO تشغيل مضخة: هو يشغّل مدخل وحدة المرحّل، وتلامس المرحّل يوصل حمل 12 فولت. on() يغلق المرحّل وoff() يفتحه.

- الأحمال: المضخة والمروحة والإضاءة ومضخات الجرعات على مصدر 12 فولت، وليس على أرجل المتحكم أبداً.
- مصباح الإنذار مصباح 12 فولت على قناة مرحّل خاصة به.

main.py

```
GPIO17 مرحّل مضخة الري على #                                  pump = OutputDevice(17)  38
GPIO22 مرحّل المروحة على #                                  fan = OutputDevice(22)  39
GPIO27 مرحّل الإضاءة على #                                light = OutputDevice(27)  40
GPIO23 مرحّل مصباح الإنذار على #                          alarm_lamp = OutputDevice(23)  41
GPIO5 مرحّل مضخة الجرعات A على #                               dose_a = OutputDevice(5)  42
GPIO6 مرحّل مضخة الجرعات B على #                               dose_b = OutputDevice(6)  43
لعرض حالة جهاز "OFF" أو "def on_off(device):                                  # "ON  204
الجهاز يعمل الآن؟ #                              :if device.is_active      205
يعمل #                                  "return "ON          206
متوقف #                                      "return "OFF      207
```

4 · الشروط والحماية

الدالة water_ok() هي المكان الوحيد الذي يقرر "هل يوجد ماء كافٍ؟". الري يفحصها قبل كل ثانية وأثناءها، ويفحص تيار المضخة: المضخة المفصولة أو العالقة لا تسحب تياراً، فيوقفها البرنامج ويتذكر العطل.

- المستوى فوق 100٪ يُعدّ قراءة غير صالحة وليس "ممتلئاً".
- تُطفأ المضخة دائماً في النهاية مهما كان سبب توقف الري.

main.py

```
def water_ok():                                    # هل يوجد ماء كافٍ؟
    return water_level >= MIN_WATER and water_level <= 100      # قراءة غير صالحة = أكثر من 100%
# <<< IRRIGATION
def irrigate():                                    # ري لمدة PUMP_SECONDS مع الحماية
    global pump_failed                             # قد نغيّر هذا المتغير العام
    print("IRRIGATION start")                      # رسالة بداية الري
    for second in range(PUMP_SECONDS):             # ثانية بعد ثانية
        read_sensors()                             # قراءة جديدة قبل كل ثانية
        if not water_ok():                         # الماء نقص أثناء الري
            # print("IRRIGATION stopped: LOW WATER")   # نخبر بالسبب
            break                                  # نخرج من الحلقة فوراً
        pump.on()                                  # تشغيل المضخة
        time.sleep(1)                              # انتظار ثانية
        read_sensors()                             # نقرأ تيار المضخة الآن
        if pump_amps < MIN_PUMP_AMPS:              # المضخة لا تسحب تياراً = لا تعمل
            pump_failed = True                     # نتذكر العطل (يظهر الإنذار)
            print("PUMP FAILURE: no current")      # رسالة العطل
            break                                  # نتوقف عن الري
    pump.off()                                     # إيقاف المضخة دائماً في النهاية
    print("IRRIGATION end")                        # رسالة نهاية الري
# >>> IRRIGATION
```

5 · وضع AUTO

كل ثانية: المروحة تتبع الحرارة بحدّين (تعمل فوق 30 °م وتتوقف تحت 27 °م — بينهما تبقى على حالها فلا تتقطع)، والإضاءة تتبع الساعة (06:00–20:00)، والري يبدأ مرة واحدة الساعة 07:00 و12:00 و17:00 إذا وُجد ماء.

- لا توجد قراءة حرارة ← تعمل المروحة (الجانب الآمن للنباتات).
- المتغير last_irrigation_hour يمنع تكرار الري خلال الساعة نفسها.

main.py

```
def auto_control():                                # الوضع التلقائي
    global last_irrigation_hour                    # سنغيّر هذا المتغير العام
    hour = time.localtime().tm_hour                # الساعة الحالية (0 إلى 23)
# >>> FAN
    if temperature is None:                        # لا قراءة حرارة: نشغّل المروحة للأمان
        fan.on()                                   # تشغيل المروحة
    elif temperature > MAX_TEMP:                   # حار جداً
        fan.on()                                   # تشغيل المروحة
    elif temperature < FAN_OFF_TEMP:               # برد بما يكفي
        fan.off()                                  # إيقاف المروحة (بين 27 و 30 تبقى كما هي)
# <<< FAN
# >>> LIGHT
    if hour >= LIGHT_ON_HOUR and hour < LIGHT_OFF_HOUR:   # داخل ساعات الإضاءة
        light.on()                                 # تشغيل الإضاءة
    else:                                          # خارج ساعات الإضاءة
        light.off()                                # إطفاء الإضاءة
# <<< LIGHT
    if hour in IRRIGATION_HOURS and hour != last_irrigation_hour:   # حان وقت ري جديد
        last_irrigation_hour = hour                # نتذكر الساعة حتى لا نكرر الري
        if water_ok():                             # يوجد ماء؟
            irrigate()                             # نروي
        else:                                      # لا يوجد ماء
            # print("IRRIGATION skipped: LOW WATER")   # نتخطى الري
```

6 · وضع DOSING

افحص الماء، أطفئ B، شغّل A لمدة (الجرعة ÷ التدفق) ثانية، أطفئ A، شغّل B، أطفئ B، اطبع DOSING DONE. لا تعمل A وB معاً أبداً. دقة الكميات تعتمد على معايرة تدفق المضخات الحقيقية (هنا 2 مل/ث) وليست مضمونة بالحساب.

- يبدأ بزر DOSING (أو /mode/DOSING من صفحة ويب Pi).
- المزرعة الذكية تعدّ المليلترات التي ضُخّت فعلاً، فيمكنك التحقق من الجرعات.

main.py

```
# >>> DOSING
def run_dosing():                                  # جرعة A ثم جرعة B (لا تعملان معاً أبداً)
    if not water_ok():                             # لا تسميد بدون ماء
        # print("DOSING cancelled: LOW WATER")     # نخبر بالسبب
        return                                     # نخرج من الدالة
```

```
A متوقفة قبل تشغيل B تأكد أن #                                    ()dose_b.off      125
A تشغيل #                                  ()dose_a.on      126
A رسالة الجرعة #              print("DOSING A", DOSE_A_ML, "ml")      127
(10 ÷ 2 = 5 ثوانٍ) الزمن = الكمية ÷ التدفق #         time.sleep(DOSE_A_ML / FLOW_ML_PER_SEC)      128
A إيقاف #                                  ()dose_a.off      129
B ثم تشغيل #                                  ()dose_b.on      130
B رسالة الجرعة #              print("DOSING B", DOSE_B_ML, "ml")      131
ثانية time.sleep(DOSE_B_ML / FLOW_ML_PER_SEC)         # 5 ÷ 2 = 2.5      132
B إيقاف #                                  ()dose_b.off      133
انتهت الجرعات #                           print("DOSING DONE")      134
135  # <<< DOSING
```

7 · وضع TEST

يعمل كل مخرج ثانيتين بالترتيب: المروحة، الإضاءة، الجرعة A، الجرعة B، ثم المضخة — والمضخة فقط إذا وُجد ماء. الحماية تبقى فعّالة في وضع الاختبار.

- استخدمه بعد توصيل مزرعة حقيقية: يجب أن ينقر كل مرحّل مرة ويعمل كل حمل.
- الجرعة A والجرعة B تعملان الواحدة بعد الأخرى، لا معاً.

main.py

```
تجربة كل جهاز لثانيتين #                                  :()def run_test  138
قائمة #  :for name, device in [("FAN", fan), ("LIGHT", light), ("DOSE A", dose_a), ("DOSE B", dose_b)]      139
↪ الأجهزة
اسم الجهاز الذي نختبره #                       print("TEST", name)          140
تشغيله #                             ()device.on          141
ننتظر ثانيتين #                    time.sleep(TEST_SECONDS)          142
إيقافه #                             ()device.off          143
(الحماية لا تُلغى) المضخة فقط إذا يوجد ماء #                                  :()if water_ok      144
نختبر المضخة #                           print("TEST PUMP")          145
تشغيلها #                                ()pump.on          146
ننتظر ثانيتين #                    time.sleep(TEST_SECONDS)          147
إيقافها #                                ()pump.off          148
لا يوجد ماء كافٍ #                                        :else      149
لا نشغّل المضخة فارغة #       print("TEST PUMP skipped: LOW WATER")          150
انتهى الاختبار #                           print("TEST DONE")      151
```

8 · الإنذارات

سلسلة if/elif واحدة تختار أهم إنذار: SENSOR ERROR ثم LOW WATER ثم PUMP FAILURE ثم HIGH TEMPERATURE ثم OUT OF RANGE، أو NONE. مصباح الإنذار يعمل مع أي إنذار، ويُطبع الإنذار فقط عند تغيّره.

- ترتيب أسطر elif هو الأولوية: الحساس المفقود أهم من ارتفاع الحرارة.
- إنذار PUMP FAILURE يبقى حتى إعادة تشغيل البرنامج (يجب أن يفحص شخص المضخة).

main.py

```
179  # >>> ALARMS
اختيار أهم إنذار وتشغيل المصباح #                                  :()def check_alarms  180
سنغيّر هذا المتغير العام #                                global alarm      181
نحفظ الإنذار السابق #                                  old = alarm      182
قراءة غير صالحة #  :if temperature is None or co2 is None or ph < 1 or ph > 14 or water_level > 100      183
خطأ حساس #                      "alarm = "SENSOR ERROR          184
الماء قليل #                             :()elif not water_ok      185
نقص الماء #                         "alarm = "LOW WATER          186
المضخة لم تسحب تياراً #                                :elif pump_failed      187
عطل المضخة #                       "alarm = "PUMP FAILURE          188
حرارة عالية #                     :elif temperature > MAX_TEMP      189
ارتفاع الحرارة #                   "alarm = "HIGH TEMPERATURE          190
خارج الحدود #  :elif ph < PH_MIN or ph > PH_MAX or ec < EC_MIN or ec > EC_MAX or co2 > CO2_MAX      191
قيمة خارج المجال #                        "alarm = "OUT OF RANGE          192
كل شيء طبيعي #                                        :else      193
لا يوجد إنذار #                              "alarm = "NONE          194
لا إنذار #                                 :"if alarm == "NONE      195
إطفاء مصباح الإنذار #                                ()alarm_lamp.off          196
يوجد إنذار #                                        :else      197
تشغيل مصباح الإنذار #                                 ()alarm_lamp.on          198
نطبع الإنذار عند تغيّره فقط #                                  :if alarm != old      199
رسالة الإنذار #                       print("ALARM", alarm)          200
201  # <<< ALARMS
```

9 · العرض والمراقبة

كل 5 ثوانٍ يُطبع سطر STATUS في الطرفية، وصفحة Flask (تبويب Web) تطلب status/ كل ثانيتين. الحلقة الرئيسية في النهاية هي البرنامج كله في 6 خطوات.

- السطر STATUS mode=... temp=... ... alarm=... day=... هو نفسه في المتحكمات الثلاثة.
- لوحة الحالة في هذه الصفحة تقرأ هذا السطر من البرنامج أثناء تشغيله.

main.py

```
# سطر الحالة في الطرفية                                    def show_status():  210
    print("STATUS mode=" + mode, "temp=" + str(temperature), "hum=" + str(humidity), "co2=" + str(co2),  # 211
الوضع والهواء
  ,ph=" + str(ph), "ec=" + str(ec), "level=" + str(water_level), "pump=" + on_off(pump)"  # الماء  212
والمضخة
          "fan=" + on_off(fan), "light=" + on_off(light), "alarm=" + alarm, "day=" + str(growth_day()))  # 213
الباقي
# ---------- صفحة الويب (Flask) ---------- 4  216
# إنشاء تطبيق الويب                                app = Flask(__name__)  217
218
219
# الصفحة الرئيسية                                          def home():  220
# الملف templates/index.html            return render_template("index.html")  221
222
223
# البيانات بصيغة JSON للصفحة                                        def status():  224
# القراءات  ,return jsonify(temperature=temperature, humidity=humidity, co2=co2, ph=ph, ec=ec  225
# الحالة  ,water_level=water_level, mode=mode, pump=on_off(pump), fan=on_off(fan)  226
# الإضاءة والإنذار واليوم  (()light=on_off(light), alarm=alarm, day=growth_day  227
228
229
# زر في الصفحة يطلب TEST أو DOSING                                  def set_mode(name):  230
# سنغيّر الوضع العام                                    global mode  231
# نقبل هذين الاسمين فقط          :"if name == "DOSING" or name == "TEST  232
# الحلقة الرئيسية تنفّذه                                    mode = name  233
# نرد بالوضع الحالي                        return jsonify(mode=mode)  234
235
236
# ربط العنوان / بالدالة home                 app.add_url_rule("/", "home", home)  237
# يعطي القراءات status/ العنوان       app.add_url_rule("/status", "status", status)  238
# العنوان mode/DOSING/ أو mode/TEST/  app.add_url_rule("/mode/<name>", "set_mode", set_mode)  239
# الويب  ()threading.Thread(target=app.run, kwargs={"host": "0.0.0.0", "port": 5000}, daemon=True).start  240
في الخلفية
# ---------- الحلقة الرئيسية (5 ---------- #  242
# رسالة البداية        print("Smart farm started (Raspberry Pi 5)")  243
# عدّاد الدورات                                          loop = 0  244
# تكرار إلى الأبد                                      :while True  245
قراءة الحساسات (read_sensors()                                  # 1  246
زر الجرعات مضغوط؟ (if dosing_button.is_pressed:                    # 2  247
# نختار وضع الجرعات                              "mode = "DOSING  248
# زر الاختبار مضغوط؟                        :if test_button.is_pressed  249
# نختار وضع الاختبار                               "mode = "TEST  250
تنفيذ الوضع (if mode == "DOSING":                                  # 3  251
# الجرعات A ثم B                                ()run_dosing  252
# ثم العودة إلى التلقائي                                "mode = "AUTO  253
# وضع الاختبار                              :"elif mode == "TEST  254
# تجربة الأجهزة                                ()run_test  255
# ثم العودة إلى التلقائي                                "mode = "AUTO  256
# الوضع AUTO                                            :else  257
# المروحة والإضاءة والري                             ()auto_control  258
الإنذارات (check_alarms()                                      # 4  259
الحالة كل 5 ثوانٍ (if loop % 5 == 0:                                  # 5  260
# سطر STATUS                                 ()show_status  261
# الدورة التالية                                loop = loop + 1  262
انتظار ثانية ثم التكرار (time.sleep(1)                                     # 6  263
```

6. البرنامج كاملاً

main.py

```
# COMPETITION MASTER 01 — Raspberry Pi 5 Complete Smart Farm
# مزرعة ذكية كاملة على Raspberry Pi 5 — نفس منطق مشروعي ESP32 و Wiren Board
import time                                    # الوقت والانتظار (sleep)
import threading                               # فقط لتشغيل صفحة الويب في الخلفية
import serial                                  # منفذ UART لقراءة حساس CO2
import adafruit_dht                            # مكتبة حساس الحرارة والرطوبة DHT22
import board                                   # أسماء أرجل اللوحة مثل board.D16
# from gpiozero import OutputDevice, Button, MCP3008  # مخارج المرحّلات + الأزرار + محوّل ADC
# from flask import Flask, jsonify, render_template   # صفحة الويب البسيطة

# ---------- 1) الإعدادات: قيم تدريبية يمكنك تغييرها ----------
MAX_TEMP = 30                # فوق 30 درجة: تشغيل المروحة
FAN_OFF_TEMP = 27            # تحت 27 درجة: إيقاف المروحة
MIN_WATER = 20               # أقل مستوى ماء مسموح (%) لتشغيل المضخة
PUMP_SECONDS = 5             # مدة الري بالثواني
# IRRIGATION_HOURS = [7, 12, 17]  # ساعات الري في اليوم
LIGHT_ON_HOUR = 6            # ساعة تشغيل الإضاءة
LIGHT_OFF_HOUR = 20          # ساعة إطفاء الإضاءة
DOSE_A_ML = 10               # جرعة السماد A بالمليلتر
DOSE_B_ML = 5                # جرعة السماد B بالمليلتر
FLOW_ML_PER_SEC = 2          # تدفق مضخة الجرعات (يحتاج معايرة على الجهاز الحقيقي)
PH_MIN = 5.5                 # أقل pH مقبول
PH_MAX = 6.5                 # أعلى pH مقبول
EC_MIN = 1.0                 # أقل EC مقبول (mS/cm)
EC_MAX = 2.5                 # أعلى EC مقبول
CO2_MAX = 1500               # أعلى CO2 مقبول (ppm)
MIN_PUMP_AMPS = 0.2          # أقل تيار يثبت أن المضخة تعمل فعلاً (أمبير)
TEST_SECONDS = 2             # مدة تشغيل كل جهاز في وضع الاختبار

# >>> PINS
# ---------- 2) التوصيلات: كل جهاز على رجله ----------
dht = adafruit_dht.DHT22(board.D16)                # DHT22 البيانات على GPIO16
co2_port = serial.Serial("/dev/serial0", 9600, timeout=1)  # MH-Z19B على UART0 (GPIO15 و GPIO14)
ph_adc = MCP3008(channel=0)                        # حساس pH على قناة ADC رقم 0
ec_adc = MCP3008(channel=1)                        # حساس EC على القناة 1
level_adc = MCP3008(channel=2)                     # حساس مستوى الماء على القناة 2
current_adc = MCP3008(channel=3)                   # حساس تيار المضخة ACS712 على القناة 3
pump = OutputDevice(17)                            # مرحّل مضخة الري على GPIO17
fan = OutputDevice(22)                             # مرحّل المروحة على GPIO22
light = OutputDevice(27)                           # مرحّل الإضاءة على GPIO27
alarm_lamp = OutputDevice(23)                      # مرحّل مصباح الإنذار على GPIO23
dose_a = OutputDevice(5)                           # مرحّل مضخة الجرعات A على GPIO5
dose_b = OutputDevice(6)                           # مرحّل مضخة الجرعات B على GPIO6
dosing_button = Button(26)                         # زر وضع الجرعات على GPIO26 (إلى GND)
test_button = Button(19)                           # زر وضع الاختبار على GPIO19 (إلى GND)
# <<< PINS

# ---------- 3) حالة النظام ----------
mode = "AUTO"                # الوضع الحالي: AUTO أو DOSING أو TEST
alarm = "NONE"               # الإنذار الحالي
pump_failed = False          # يصبح True إذا شغّلنا المضخة ولم يمر تيار
temperature = None           # آخر قراءة حرارة
humidity = None              # آخر قراءة رطوبة
co2 = None                   # آخر قراءة CO2
ph = 0                       # آخر قراءة pH
ec = 0                       # آخر قراءة EC
water_level = 0              # مستوى الماء %
pump_amps = 0                # تيار المضخة بالأمبير
last_irrigation_hour = -1    # آخر ساعة تم فيها الري (حتى لا نكرره)
start_time = time.time()     # وقت بدء التشغيل لحساب يوم النمو
last_dht_time = 0            # وقت آخر قراءة DHT22 (يحتاج ثانيتين بين القراءات)


def read_co2():                                    # قراءة CO2 من MH-Z19B عبر UART
    co2_port.write(bytes([0xFF, 0x01, 0x86, 0, 0, 0, 0, 0, 0x79]))  # أمر طلب القراءة (9 بايت)
    reply = co2_port.read(9)                       # ننتظر رداً من 9 بايت
    if len(reply) != 9:                            # لا يوجد رد = الحساس مفصول
        return None                                # None تعني: لا توجد قراءة
    return reply[2] * 256 + reply[3]               # البايت 2 و 3 = التركيز ppm


```

```
# >>> SENSORS
# قراءة كل الحساسات                                 :()def read_sensors
# سنغيّر هذه المتغيرات  global temperature, humidity, co2, ph, ec, water_level, pump_amps, last_dht_time
↪ العامة
if time.time() - last_dht_time >= 2:  قراءة جديدة كل ثانيتين فقط            # DHT22
# نحفظ وقت هذه القراءة                  ()last_dht_time = time.time
# نحاول القراءة                                      :try
# الحرارة °C           temperature = dht.temperature
# الرطوبة %                humidity = dht.humidity
# فشل القراءة (الحساس مفصول)                        :except RuntimeError
# لا توجد قراءة حرارة                    temperature = None
# ولا رطوبة                           humidity = None
co2 = read_co2()  (أو None)                             # CO2 ppm
ph = round(ph_adc.voltage * 3.5, 2)  الجهد × 3.5          # pH = 3.5
ec = round(ec_adc.voltage / 0.5, 2)  الجهد ÷ 0.5          # EC
water_level = round(level_adc.voltage / 3.0 * 100)  # 3  فولت = %100
pump_amps = round((current_adc.voltage - 2.5) / 0.185, 2)  # ACS712: 2.5  فولت = صفر أمبير
# <<< SENSORS


# هل يوجد ماء كافٍ؟                                  :()def water_ok
# return water_level >= MIN_WATER and water_level <= 100  أكثر من %100 = قراءة غير صالحة


# يوم النمو منذ بدء التشغيل                                  :()def growth_day
return int((time.time() - start_time) / 86400) + 1  # 86400  ثانية = يوم واحد


# >>> IRRIGATION
# ري لمدة PUMP_SECONDS مع الحماية                                    :()def irrigate
# قد نغيّر هذا المتغير العام                              global pump_failed
# رسالة بداية الري                        print("IRRIGATION start")
# ثانية بعد ثانية                :for second in range(PUMP_SECONDS)
# قراءة جديدة قبل كل ثانية                                ()read_sensors
# الماء نقص أثناء الري                           :()if not water_ok
# نخبر بالسبب  print("IRRIGATION stopped: LOW WATER")
# نخرج من الحلقة فوراً                                      break
# تشغيل المضخة                                    ()pump.on
# انتظار ثانية                                  time.sleep(1)
# نقرأ تيار المضخة الآن                                 ()read_sensors
# المضخة لا تسحب تياراً = لا تعمل                :if pump_amps < MIN_PUMP_AMPS
# نتذكر العطل (يظهر الإنذار)                        pump_failed = True
# رسالة العطل       print("PUMP FAILURE: no current")
# نتوقف عن الري                                    break
# إيقاف المضخة دائماً في النهاية                                        ()pump.off
# رسالة نهاية الري                         print("IRRIGATION end")
# <<< IRRIGATION


# >>> DOSING
# جرعة A ثم جرعة B (لا تعملان معاً أبداً)                                    :()def run_dosing
# لا تسميد بدون ماء                                :()if not water_ok
# نخبر بالسبب        print("DOSING cancelled: LOW WATER")
# نخرج من الدالة                                        return
# تأكد أن B متوقفة قبل تشغيل A                                  ()dose_b.off
# تشغيل A                                       ()dose_a.on
# رسالة الجرعة A              print("DOSING A", DOSE_A_ML, "ml")
# الزمن = الكمية ÷ التدفق (10 ÷ 2 = 5 ثوانٍ)         time.sleep(DOSE_A_ML / FLOW_ML_PER_SEC)
# إيقاف A                                       ()dose_a.off
# ثم تشغيل B                                        ()dose_b.on
# رسالة الجرعة B              print("DOSING B", DOSE_B_ML, "ml")
time.sleep(DOSE_B_ML / FLOW_ML_PER_SEC)  ثانية         # 5 ÷ 2 = 2.5
# إيقاف B                                       ()dose_b.off
# انتهت الجرعات                                print("DOSING DONE")
# <<< DOSING


# تجربة كل جهاز لثانيتين                                        :()def run_test
# قائمة الأجهزة  :for name, device in [("FAN", fan), ("LIGHT", light), ("DOSE A", dose_a), ("DOSE B", dose_b)]
# اسم الجهاز الذي نختبره                           print("TEST", name)
# تشغيله                                  ()device.on
# ننتظر ثانيتين                        time.sleep(TEST_SECONDS)
# إيقافه                                  ()device.off
# المضخة فقط إذا يوجد ماء (الحماية لا تُلغى)                                    :()if water_ok
# نختبر المضخة                             print("TEST PUMP")
# تشغيلها                                      ()pump.on
```

```
147 ننتظر ثانيتين #                    time.sleep(TEST_SECONDS)
148 إيقافها #                                 ()pump.off
149 لا يوجد ماء كافٍ #                                    :else
150 لا نشغّل المضخة فارغة #       print("TEST PUMP skipped: LOW WATER")
151 انتهى الاختبار #                              print("TEST DONE")
152
153
154 الوضع التلقائي #                                 :()def auto_control
155 سنغيّر هذا المتغير العام #                    global last_irrigation_hour
156 (23 إلى 0) الساعة الحالية #                 hour = time.localtime().tm_hour
157     # >>> FAN
158 لا قراءة حرارة: نشغّل المروحة للأمان #                        :if temperature is None
159 تشغيل المروحة #                                   ()fan.on
160 حار جداً #                    :elif temperature > MAX_TEMP
161 تشغيل المروحة #                                   ()fan.on
162 برد بما يكفي #                :elif temperature < FAN_OFF_TEMP
163 (بين 27 و 30 تبقى كما هي) إيقاف المروحة #                                ()fan.off
164     # <<< FAN
165     # >>> LIGHT
166 داخل ساعات الإضاءة #  :if hour >= LIGHT_ON_HOUR and hour < LIGHT_OFF_HOUR
167 تشغيل الإضاءة #                                    ()light.on
168 خارج ساعات الإضاءة #                                       :else
169 إطفاء الإضاءة #                                  ()light.off
170     # <<< LIGHT
171 حان وقت ري جديد #  :if hour in IRRIGATION_HOURS and hour != last_irrigation_hour
172 نتذكر الساعة حتى لا نكرر الري #                 last_irrigation_hour = hour
173 يوجد ماء؟ #                              :()if water_ok
174 نروي #                               ()irrigate
175 لا يوجد ماء #                                    :else
176 نتخطى الري #  print("IRRIGATION skipped: LOW WATER")
177
178
179 # >>> ALARMS
180 اختيار أهم إنذار وتشغيل المصباح #                                 :()def check_alarms
181 سنغيّر هذا المتغير العام #                                   global alarm
182 نحفظ الإنذار السابق #                                  old = alarm
183 قراءة غير صالحة #  :if temperature is None or co2 is None or ph < 1 or ph > 14 or water_level > 100
184 خطأ حساس #                       "alarm = "SENSOR ERROR
185 الماء قليل #                              :()elif not water_ok
186 نقص الماء #                          "alarm = "LOW WATER
187 المضخة لم تسحب تياراً #                                :elif pump_failed
188 عطل المضخة #                        "alarm = "PUMP FAILURE
189 حرارة عالية #                      :elif temperature > MAX_TEMP
190 ارتفاع الحرارة #                    "alarm = "HIGH TEMPERATURE
191 خارج الحدود #  :elif ph < PH_MIN or ph > PH_MAX or ec < EC_MIN or ec > EC_MAX or co2 > CO2_MAX
192 قيمة خارج المجال #                       "alarm = "OUT OF RANGE
193 كل شيء طبيعي #                                    :else
194 لا يوجد إنذار #                                "alarm = "NONE
195 لا إنذار #                                :"if alarm == "NONE
196 إطفاء مصباح الإنذار #                                ()alarm_lamp.off
197 يوجد إنذار #                                       :else
198 تشغيل مصباح الإنذار #                                ()alarm_lamp.on
199 نطبع الإنذار عند تغيّره فقط #                                  :if alarm != old
200 رسالة الإنذار #                         print("ALARM", alarm)
201 # <<< ALARMS
202
203
204 لعرض حالة جهاز "OFF" أو "def on_off(device):                                    # "ON
205 الجهاز يعمل الآن؟ #                               :if device.is_active
206 يعمل #                                "return "ON
207 متوقف #                                    "return "OFF
208
209
210 سطر الحالة في الطرفية #                                    :()def show_status
211     print("STATUS mode=" + mode, "temp=" + str(temperature), "hum=" + str(humidity), "co2=" + str(co2),  #
↪ الوضع والهواء
212 الماء #  ,ph=" + str(ph), "ec=" + str(ec), "level=" + str(water_level), "pump=" + on_off(pump)"
↪ والمضخة
213           "fan=" + on_off(fan), "light=" + on_off(light), "alarm=" + alarm, "day=" + str(growth_day()))  #
↪ الباقي
214
215
216 ---------- (Flask) صفحة الويب (4 ---------- #
217 إنشاء تطبيق الويب #                                  app = Flask(__name__)
218
219
220 الصفحة الرئيسية #                                       :()def home
```

```
templates/index.html الملف #            return render_template("index.html")


للصفحة JSON البيانات بصيغة #                                        :()def status
القراءات #  ,return jsonify(temperature=temperature, humidity=humidity, co2=co2, ph=ph, ec=ec
الحالة #  ,water_level=water_level, mode=mode, pump=on_off(pump), fan=on_off(fan)
الإضاءة والإنذار واليوم #  (()light=on_off(light), alarm=alarm, day=growth_day


TEST أو DOSING زر في الصفحة يطلب #                                  :def set_mode(name)
سنغيّر الوضع العام #                                    global mode
نقبل هذين الاسمين فقط #          :"if name == "DOSING" or name == "TEST
الحلقة الرئيسية تنفّذه #                                    mode = name
نرد بالوضع الحالي #                        return jsonify(mode=mode)


home ربط العنوان / بالدالة #                 app.add_url_rule("/", "home", home)
يعطي القراءات status/ العنوان #       app.add_url_rule("/status", "status", status)
mode/TEST/ أو mode/DOSING/ العنوان #  app.add_url_rule("/mode/<name>", "set_mode", set_mode)
الويب في #  ()threading.Thread(target=app.run, kwargs={"host": "0.0.0.0", "port": 5000}, daemon=True).start
الخلفية


---------- الحلقة الرئيسية (5 ---------- #
رسالة البداية #        print("Smart farm started (Raspberry Pi 5)")
عدّاد الدورات #                                          loop = 0
تكرار إلى الأبد #                                       :while True
قراءة الحساسات (read_sensors()                                  # 1
زر الجرعات مضغوط؟ (if dosing_button.is_pressed:                    # 2
نختار وضع الجرعات #                            "mode = "DOSING
زر الاختبار مضغوط؟ #                       :if test_button.is_pressed
نختار وضع الاختبار #                             "mode = "TEST
تنفيذ الوضع (if mode == "DOSING":                                # 3
B ثم A الجرعات #                                ()run_dosing
ثم العودة إلى التلقائي #                               "mode = "AUTO
وضع الاختبار #                               :"elif mode == "TEST
تجربة الأجهزة #                                ()run_test
ثم العودة إلى التلقائي #                               "mode = "AUTO
AUTO الوضع #                                           :else
المروحة والإضاءة والري #                              ()auto_control
الإنذارات (check_alarms()                                  # 4
الحالة كل 5 ثوانٍ (if loop % 5 == 0:                                  # 5
STATUS سطر #                                 ()show_status
الدورة التالية #                                loop = loop + 1
انتظار ثانية ثم التكرار (time.sleep(1)                                  # 6

```

templates/index.html

```
<!doctype html>
<html lang="ar" dir="rtl"><head><meta charset="utf-8"><meta name="viewport" content="width=device-width,
initial-scale=1">
<title>Smart Farm — Raspberry Pi 5</title>
<style>
body{font-family:system-ui;margin:14px;background:#f2f6f3}
.grid{display:grid;grid-template-columns:repeat(auto-fill,minmax(120px,1fr));gap:8px}
.c{background:#fff;border-radius:10px;padding:8px;box-shadow:0 1px 3px #0002}.c b{display:block;font-size:22px}
button{font-size:16px;padding:8px 14px;margin:6px 4px}#alarm{font-weight:700;margin:10px 0}
</style></head>
<body>
<Raspberry Pi 5</h2 — المزرعة الذكية 🌱<h2>
<div class="grid">
<b id="humidity">--</b></div>% الرطوبة<"C<b id="temperature">--</b></div><div class="c° الحرارة<"div class="c>
 <div class="c">CO2 ppm<b id="co2">--</b></div><div class="c">pH<b id="ph">--</b></div>
<b id="water_level">--</b></div>% مستوى الماء<"div class="c">EC<b id="ec">--</b></div><div class="c>
<b id="day">--</b></div>يوم النمو<"b id="mode">--</b></div><div class="c>الوضع<"div class="c>
<b id="fan">--</b></div>المروحة<"b id="pump">--</b></div><div class="c>المضخة<"div class="c>
<b id="light">--</b></div>الإضاءة<"div class="c>
</div>
<div id="alarm">--</div>
<button/>تشغيل الجرعات<"button onclick="fetch('/mode/DOSING')>
<button/>وضع الاختبار<"button onclick="fetch('/mode/TEST')>
<script>
function update(){fetch('/status').then(function(r){return r.json()}).then(function(s){
 for (var k in s){var e=document.getElementById(k); if(e) e.textContent = s[k]===null ? '--' : s[k];}
{;({;s.alarm + ' :الإنذار' = document.getElementById('alarm').textContent
update(); setInterval(update, 2000);
</script></body></html>
```

```
29
```

7. سيناريوهات الاختبار ونتائجها

نتائج السيناريوهات التي شُغّلت في المتصفح بتاريخ 05-11-2026 (بيئة التشغيل الحقيقية في المحاكي).

السيناريو	ماذا يفعل	المتوقع	النتيجة
1 · التشغيل العادي	الساعة 08:00، الحرارة 25 °م، الماء 75٪: شغّل 12 ثانية دون أي تدخل.	أسطر STATUS بكل القراءات، الوضع AUTO، الإضاءة تعمل (06–20)، المروحة والمضخة متوقفتان، الإنذار NONE، مصباح الإنذار مطفأ.	PASS
2 · ارتفاع الحرارة (25 ← 35 °م)	عند 3 ث تصبح الحرارة 35 °م، وعند 9 ث تصبح 28.5 °م (بين الحدين)، وعند 15 ث تصبح 25 °م.	تعمل المروحة ويظهر إنذار HIGH TEMPERATURE فوق 30 °م؛ عند 28.5 °م يزول الإنذار وتبقى المروحة تعمل (هامش التخلّف)؛ وتحت 27 °م تتوقف المروحة.	PASS
3 · انخفاض الماء	الساعة 11:59:55 — عند 2 ث ينخفض الخزان إلى 10٪؛ وموعد ري 12:00 بعد 3 ثوانٍ.	إنذار LOW WATER، مصباح الإنذار يعمل، و"IRRIGATION skipped: LOW WATER": المضخة لا تعمل أبداً.	PASS
4 · الري الساعة 12:00	ابدأ الساعة 11:59:56 وشغّل 25 ثانية.	"IRRIGATION start"، تعمل المضخة نحو 5 ثوانٍ (مع قياس التيار كل ثانية)، ثم "IRRIGATION end"، ومرة واحدة فقط في تلك الساعة.	PASS
5 · الجرعات A ثم B	عند 2 ث اضغط زر DOSING لمدة 1.5 ثانية.	DOSING A 10 مل (5 ث)، ثم DOSING B 5 مل (2.5 ث)، ولا تعملان معاً أبداً، ثم DOSING DONE والعودة إلى AUTO. الكميات من نموذج المضخات في المزرعة (2 مل/ث).	PASS
6 · وضع الاختبار	الساعة 21:00 (الإضاءة مطفأة حسب الجدول): عند 2 ث اضغط زر TEST لمدة 1.5 ثانية.	TEST FAN ثم TEST LIGHT ثم TEST DOSE A ثم TEST DOSE B ثم TEST PUMP (لأن الماء كافٍ فقط)، ثم TEST DONE؛ يعمل كل حمل مرة واحدة، ثم يطفئ AUTO الإضاءة من جديد.	PASS
7 · عطل حساس	عند 3 ث يُضبط عطل "Sensor disconnected" على حساس الهواء (DHT22)؛ وعند 10 ث يُزال العطل.	إنذار SENSOR ERROR، وقيمة temp=None/null في STATUS، وتعمل المروحة (الجانب الآمن) ما دامت القراءة مفقودة؛ ويزول الإنذار بعد الإصلاح.	PASS
8 · جدول الإضاءة	ابدأ الساعة 19:59:50 وشغّل 18 ثانية.	الإضاءة تعمل قبل 20:00 وتنطفئ بعد 20:00 (LIGHT_OFF_HOUR).	PASS
9 · خطأ توصيل (سلك المضخة مفصول)	يُزال السلك من تلامس المرحل NO1 إلى ACS712، ثم يبدأ ري 12:00 (البداية 11:59:56).	يعمل المرحل لكن لا يمر تيار: "PUMP FAILURE: no current"، وإنذار PUMP FAILURE، ويُطفأ المرحل فوراً.	PASS
10 · النظام الكامل	الساعة 11:59:52: ري الساعة 12:00، وعند 14 ث تصل الحرارة إلى 35 °م، وعند 18 ث اضغط زر DOSING لمدة 1.5 ثانية، وعند 32 ث تعود الحرارة إلى 25 °م.	ري (نحو 5 ث)، ثم المروحة وإنذار HIGH TEMPERATURE، ثم الجرعات A ثم B، ثم تزول كل الإنذارات وتتوقف المروحة: يعود النظام إلى AUTO العادي.	PASS

8. غيّر شيئاً واحداً — تمارين

1. غيّر حد تشغيل المروحة

   اجعل MAX_TEMP = 28 وFAN_OFF_TEMP = 25. شغّل، ثم اضبط حرارة هواء المزرعة على 29 °م.

   ما يجب أن تلاحظه: تعمل المروحة الآن عند 29 °م ويظهر إنذار HIGH TEMPERATURE.

2. غيّر مدة الري

   اجعل PUMP_SECONDS = 8 وأضف الساعة الحالية إلى IRRIGATION_HOURS. شغّل وراقب المضخة.

   ما يجب أن تلاحظه: يظهر IRRIGATION start ثم IRRIGATION end بعد نحو 8 ثوانٍ؛ ويُظهر تدفق المزرعة عمل المضخة.

3. انقل المروحة إلى GPIO آخر

   انقل السلك من GPIO22 (الرجل 15) إلى GPIO24 (الرجل 18) وغيّر OutputDevice(22) إلى OutputDevice(24).

   ما يجب أن تلاحظه: تعمل المروحة من الرجل الجديدة. إذا تغيّر الكود فقط أو السلك فقط تبقى المروحة متوقفة بينما يظن الكود أنها تعمل (GPIO22).

4. افصل حساس المستوى

   أثناء عمل البرنامج احذف سلك إشارة حساس مستوى الماء.

   ما يجب أن تلاحظه: يُقرأ المستوى بشكل خاطئ (قريباً من 0): إنذار LOW WATER، يضيء المصباح، ويُرفض الري والجرعات. أعد السلك (Ctrl+Z).

5. غيّر حجم الجرعة

   اجعل DOSE_A_ML = 20 وDOSE_B_ML = 10، ثم ابدأ DOSING.

   ما يجب أن تلاحظه: تعمل A 10 ثوانٍ وB 5 ثوانٍ؛ وتُظهر المزرعة نحو 20 مل و10 مل. تغيير FLOW_ML_PER_SEC دون تغيير المضخة يغيّر الزمن فقط وليس التدفق الحقيقي — لذلك المعايرة مهمة.

6. عطّل المضخة أثناء التشغيل

   اختر المضخة الرئيسية، واضبط عطلها على "Pump failure"، وابدأ الري (ضع الساعة الحالية في IRRIGATION_HOURS).

ما يجب أن تلاحظه: يغلق المرحّل لكن ACS712 لا يقيس تياراً: PUMP FAILURE: no current، وإنذار PUMP FAILURE.

7. غيّر وقت الإضاءة

اجعل LIGHT_ON_HOUR = 8 وLIGHT_OFF_HOUR = 18. استخدم الساعة (التسريع) لتجاوز 18:00.

ما يجب أن تلاحظه: تنطفئ الإضاءة الساعة 18:00 بدلاً من 20:00.

8. ضيّق مجال pH

اجعل PH_MAX = 6.0 (محلول المزرعة نحو 6.2 pH).

ما يجب أن تلاحظه: يظهر ALARM OUT OF RANGE ويضيء المصباح؛ ولا يتغير شيء آخر لأن OUT OF RANGE تحذير فقط.

9. التنفيذ على العتاد الحقيقي

1. فعّل SPI والمنفذ التسلسلي (مع تعطيل تسجيل الدخول عليه) من sudo raspi-config ← Interface Options.
2. ثبّت المكتبات (أدناه) في بيئة افتراضية: python3 -m venv --system-site-packages farm ثم farm/bin/activate ..
3. وصّل والـPi مطفأ. وحدات المرحّل: VCC من رجل 5 فولت؛ وإشارة GPIO بجهد 3.3 فولت — اختر وحدات تعمل بـ3.3 فولت (تعمل بـHIGH كما في الكود).
4. يحتاج MH-Z19B تغذية 5 فولت لكن UART الخاص به 3.3 فولت: TX→GPIO15 (RX)، وGPIO14←RX (TX).
5. وحدات pH وEC: عايِرها بمحاليل قياسية واستبدل المعاملين 3.5 و0.5 بنتيجة المعايرة.
6. انسخ main.py و templates/index.html، ثم شغّل python3 main.py وافتح http://<عنوان-5000:pi>.

10. المكتبات المطلوب تثبيتها

- gpiozero (+ lgpio on Pi 5): sudo apt install python3-gpiozero python3-lgpio
- pyserial: sudo apt install python3-serial
- adafruit-circuitpython-dht: pip install adafruit-circuitpython-dht
- Flask: sudo apt install python3-flask
- MCP3008 (gpiozero, SPI): sudo raspi-config → Interface Options → SPI → enable

11. الفروقات بين المحاكاة والجهاز الحقيقي

- الحساسات وADC والمرحّلات نماذج محاكاة؛ معادلاتها خاصة بوحدات المحاكي وليست حساساتك الحقيقية.
- صفحة Flask تُخدم داخل المحاكي (تبويب Web)؛ وعلى Pi تكون خادماً حقيقياً على المنفذ 5000.
- لا يفشل DHT22 عشوائياً هنا؛ على الجهاز الحقيقي خطأ قراءة عرضي أمر طبيعي (قد يومض الإنذار).
- المضخة والمروحة ومضخات الجرعات نماذج مقاومة؛ لا يُحاكى تيار البدء ولا الجهد العكسي — استخدم وحدات مرحّل فيها حماية.

هذه المشاريع مراجع تدريبية؛ ولا تثبت المعدات التي ستُستخدم في النهائي ولا تغطي كل الوحدات التنافسية.